



## Installation Manual

### ConfluenceGuessr

ConfluenceGuessr is a Forge app for Confluence that delivers a knowledge-discovery, interactive guessing game.

### How to get the app running

Sections 1 and 2 will be entirely done in a terminal. So, please git clone this repo in the terminal of an IDE and run the commands below in the terminal of that IDE.

#### 1. Setup (before deploying the app to your Confluence Dev site)

1.1 If you've never run forge commands then you should set up your machine for forge app development; you will have to install/deploy this app on your own confluence developer site (that you create yourself) as only the source code is handed over.

You only have to follow the steps in the link below to the end of step 2 under the heading **Build your first Forge app**. [▲ Getting started with Forge](#)

1.2 Run `forge register` to create a Forge app ID that uniquely identifies the version of ConfluenceGuessr on your development site.

You will be asked to "...select the Developer Space you want to assign your app to". Create a new developer space and name it whatever you want e.g. Test. Then you will be asked to "Enter a name for the app". Name it whatever you want e.g.

ConfluenceGuessr. Agree to the Atlassian Developer Terms and the Privacy Policy by typing y and pressing enter.

**Nb:** The manifest.yml file we hand over has our app ID removed. Before deploying, by first running:

```
forge register
```

we create a new app under your Atlassian account with the new ID in manifest.yml that originally had an empty value for the app ID. You can only do this once.

Check in the manifest.yml file that the app id isn't empty anymore.

before

```
1 app:
2   runtime:
3     name: nodejs24.x
4     memoryMB: 512
5     architecture: arm64
6   id:
7
```

after

```
1 app:
2   runtime:
3     name: nodejs24.x
4     memoryMB: 512
5     architecture: arm64
6     id: {your_app_ID}
7
```

### 1.3 Setting up the API keys

#### To get the API keys you have to do this:

**Groq API key** - Create an account with Groq Cloud Console. Pressed Create API Key and copied the key in a safe place e.g. a password manager.

**Gemini API key** - Sign up to Google AI Studio and create an API key. Used the Gemini API key instead of the default Gemini API Key. Copied the key into a safe place again.

The two AI provider API keys are stored as encrypted Forge environment variables and not in our source code. Set them once per environment by running the commands below.

You will be prompted to paste the values for both. Repeat for each environment you deploy to as these variables are not shared between environments.

#### Then run these commands:

1. `forge variables set --encrypt GROQ_API_KEY --environment {environment_name e.g. development}` (you will then be prompted to enter the actual value for the key i.e. Enter the variable value: [hidden])
2. `forge deploy` (Tip: Changes to environment variables will not apply to existing deployments. Please run `forge deploy` for your changes to take effect.)
3. `forge install --upgrade` (may have to run if prompted)
4. `forge variables set --encrypt GEMINI_API_KEY --environment {environment_name e.g. development}` (you will then be prompted to enter the actual value for the key i.e. Enter the variable value: [hidden])
5. `forge deploy` (Tip: Changes to environment variables will not apply to existing deployments. Please run `forge deploy` for your changes to take effect.)
6. `forge install --upgrade` (may have to run if prompted)

Nb: `forge variables list` will list all the environment variables that have been set. You should see both API keys.

`forge variables unset {API_KEY} -e {environment_name}` will remove a specific environment variable (API KEY in this case) from {environment\_name}. Hopefully this won't have to be used in just getting the app to run.

## 2. Deploying the app to your Confluence Dev site

### Order of operations

1. Install backend and frontend dependencies
2. Build the frontend
3. Deploy and install on Confluence dev site

The app contains a backend and a frontend that are installed separately; the frontend must be built before the first deploy. Running the steps out of order will result in a failed deployment.

Note: If the app loads, but the question generation fails (when you either start or create a game (solo mode or multiplayer mode, respectively)), check the API keys are set for that environment.

### 2.1 Installing frontend dependencies

There are a total of two package.json files and both of them need installing. From the project root, run the following commands:

```
1 npm install
2 cd static
3 npm install
4 cd ..
5
```

Note: If `npm install` fails with a peer dependency error, run it with `--legacy-peer-deps`. This is caused by a version conflict between the Forge storage library and the frontend build. It does not affect the running app though.

### 2.2 Building frontend

The manifest.yml points the app's user interface at static/build

resources:

- key: main path: static/build

That folder is a build artifact and is excluded from the repository and does not arrive with the source code. It must be generated before the first deploy by running these commands:

```
1 cd static
2 npm run build
3 cd ..
4
```

Note: If this step is skipped for whatever reason, `forge deploy` (or `forge tunnel`) fails with an error stating that `index.html` is missing from `static/build`. The fix is to run the build, not to change the manifest.

### 2.3 Deploying and installing the app

Deploy the code and then install it onto your Confluence dev site by running these commands:

```
1 forge deploy
2 forge install
3
```

When prompted, select Confluence as the Atlassian app or platform tool and enter your dev site URL. You will be asked to approve the permissions listed in the manifest.yml file.

`forge install` is only needed once per site. After that, `forge deploy` alone is sufficient to push new code and the site picks up the changes automatically (this is useful if further development is to be done to ConfluenceGuessr).

## Environments

Forge provides three environments by default: development, staging and production. Each holds its own copy of the code and its own environment variables.

For the purposes of simply getting the app to run, just let forge deploy it on the default environment (most likely development) and make sure to set the API key environment variables for that environment as the keys aren't global between environments.

## 3. Booting up the app

Now that the app has been deployed and installed on your Confluence dev site, you can simply navigate to your site. There should be an **Apps** button on the left sidebar. If you click that and then click **ConfluenceGuessr** under your apps, the ConfluenceGuessr app should boot up and start itself automatically. The app looks best on a desktop monitor due to the bigger screen size. If you're on a laptop we recommend playing it in 75%-80% normal screen size; you can minimise the screen by pressing ctrl + minus (-) on windows and cmd + minus (-) on mac, and vice versa.

## Folder structure

```
1 confluenceguessr/
2 |   src/
3 |   |   index.js           # Backend functions & resolvers
4 |   |
5 |   static/               # Frontend, Bundled assets (Custom UI
6 |   |   only)
7 |   |   |   src
8 |   |   |   |   Components # Main HTML wrapper for the UI
9 |   |   |   |   Screens    # E.g. Buttons
10 |   |   |   |   Data       # E.g. CreateGame, QuestionScreen
11 |   |   |   |   |   questions.js # Has example questions and answers
12 |   |   |   |   |   for testing in sprint 1
13 |   |   |   |   |   App.js    # Main App
14 |   |   |   |   |   App.css   # App styling
15 |   |   |   |   |   manifest.yml # App blueprint and permissions
16 |   |   |   |   |   package.json # Project dependencies
17 |   |   |   |   |   README.md  # Project documentation
```

## Automated tests

- Backend resolvers ( `src/index.js` ) are unit tested with Node's built-in test runner: `npm test` from the project root.
- Frontend components/screens ( `static/src/` ) are unit/component tested with Jest + React Testing Library: `npm test` from the `static/` folder. `@forge/bridge`'s `invoke` is mocked in every test, so these tests verify each screen's own logic and rendering in isolation.

## Why we don't have automated end-to-end tests?

True end-to-end automation (e.g. driving a real browser through the installed app) isn't practical for this project, because the app only runs meaningfully inside Forge's environment:

- **Real Confluence data and permissions.** Core behaviour ( `getAccessiblePages` , `getUsersPages` , page/space restrictions) depends on the real spaces, pages, and permission grants on a live Atlassian site. There's no way to fabricate this locally, it requires an actual Confluence instance with real content and real per-user access controls.
- **Real Forge storage (KVS).** Team games, leaderboards, and scores are persisted via `@forge/kvs` , which only exists once the app is deployed to a Forge environment. It can't be run or seeded locally.
- **Real Forge auth context.** Every resolver reads `req.context.accountId` from Forge's own auth layer. Multiplayer features ( `createTeamGame` , `submitGameScore` , `deleteGame` 's ownership check, `getCurrentUser` ) can only be verified end-to-end with genuine, distinct logged-in Atlassian accounts.
- **Client confirmation.** We raised this with our client, who confirmed that end-to-end testing for Forge apps like this is typically done with Atlassian's own internal tooling, which isn't available to us as external developers.

Instead, we rely on unit/component tests for logic correctness (above) and the manual test plan below for integration behaviour that only shows up on a real deployment.

## Manual end-to-end test plan

### Setup

1. Deploy and install the app on your developer site as described above, using the account you'll call whatever you want, say **User A**.
2. Invite a second account to the same site, let's say **User B**, so multiplayer/leaderboard/permission behaviour can be checked from more than one identity:
  - Go to your site's admin console at `https://<your-site>.atlassian.net/admin` → **Products** → **Confluence** → **Product access**, or from within Confluence go to the space's

### **Space settings → Permissions.**

- Invite a second email address (a personal email or a second test account works) and grant it access to Confluence.
  - Accept the invite from that second account so it's an active user on the site.
3. Create a Confluence space with several pages, and restrict at least one page so User B cannot see it:
- As User A, create or open a space, then create a few pages with distinct content.
  - Open one page → ... (more actions) → **Restrictions** → restrict viewing to only User A (or a group User B isn't in). Save.
  - Confirm restriction worked by logging in as User B and checking that page is not visible/accessible in Confluence itself, before testing the app.
4. Confirm both User A and User B can open the app from the Confluence left navigation (**Apps** → **ConfluenceGuessr**).

#### **1. Solo game: happy path**

1. As User A, open the app. Confirm the main menu loads with the accessible page list.
2. Leave game mode on "Solo", set number of questions, and start a game.
3. Answer at least one question correctly and one incorrectly before the timer runs out.
4. Confirm: correct answers show a points value that decreases the slower you answer; incorrect answers show 0 points; the running score total updates after each question.
5. Let the timer expire on one question without answering. Confirm the question auto-reveals the answer instead of getting stuck.
6. Finish the game and confirm the end screen shows your final score and the solo leaderboard.

#### **2. Accessibility options**

1. Start a new solo game with hints turned on. Use a hint on one question and confirm the point deduction is shown and applied.
2. Start another game with the timer and timed points turned off. Confirm no countdown is shown and a correct answer always earns full points regardless of how long you take.

#### **3. Permissions: respecting page access**

1. As User A, confirm the page-selection list includes every page they have access to in the test space, and excludes the page restricted from them (if any).
2. As User B, open the app and confirm their page list differs from User A's, specifically, that the page restricted to User B does not appear, and no restricted content is used to generate questions for them.

#### 4. Multiplayer: create, join, and score

1. As User A, switch game mode to "Multiplayer", select pages, and create a quiz. Confirm a quiz/game ID is generated and shown.
2. As User A, open "Find Game" and confirm the newly created quiz appears in the list, with User A's real display name (not a placeholder) shown under "Created by".
3. As User B, open "Find Game" and confirm the same quiz is visible (assuming shared page access) and join it.
4. As User B, complete the quiz. Confirm the score submits without error.
5. As User A, open the quiz's leaderboard from "Find Game" and confirm both users' scores appear, correctly ranked highest to lowest.
6. As User B, replay the same quiz with a lower score than their first attempt. Confirm the leaderboard keeps their higher score rather than overwriting it with the lower one.

#### 5. Multiplayer: ownership and deletion\*\*

1. As User B, confirm they cannot see a delete option on the quiz User A created.
2. As User A, delete the quiz they created from "Find Game". Confirm it disappears from the list for both User A and User B, and its leaderboard entries are also removed.

#### 6. Question review

1. Complete a solo game, then choose to review questions from the end screen.
2. Confirm each question shows the original prompt, the correct answer, and (if answered) your submitted guess, and that navigating between questions works.